

Claude Code 搭建智能体实战

李代波



前情提要

智能体原理

- I. 智能体由 LLM 加上一套运行环境和工具来构成
- I. 用 API 搭建简单的智能体
- I. 生活中常见的 Agent

Claude Code 使用

- I. 下载、安装和运行 Claude Code
- I. 创建 Skill 和 MCP 服务器
- I. 搭建简易科研智能体

亲手搭建一个 Agent

- I. 直接将 Claude Code 变成你的 Agent
- I. 用 Python 做更强的控制
- I. 丰富的多智能体!

注：本课件中 Claude Code 接入的模型是国产的 Minimax-M2.7

实验一

关于搭建 Survey 智能体

——但是先做 Survey 再搭 Agent

Part I

将 Claude Code 变成专业的 Agent

让 Claude Code 写一篇综述

- 初次给定 prompt:
 - 围绕机器学习，注意力机制，神经网络优化等领域，展开调研，写成一篇综述，tex 大概不要写太短，必须有 bibtex
- 右图问题：
 - 搜索工具失效
 - 在没有搜索结果的情况下依然继续写综述

围绕机器学习，注意力机制，神经网络优化等领域，展开调研，写成一篇综述，tex 大概不要写太短，必须有 bibtex

- 我来为你调研并撰写一篇关于机器学习、注意力机制和神经网络优化的综述。首先让我检查一下当前环境和 memory。

Read 1 file, listed 1 directory (ctrl+o to expand)

- Web Search("attention mechanism neural network deep learning survey 2024 2025")
 - └ Did 0 searches in 118ms

- Web Search("neural network optimization methods survey Adam optimizer 2024")
 - └ Did 0 searches in 105ms

- Web Search("transformer attention mechanism self-attention recent advances 2025")
 - └ Did 0 searches in 78ms

- Web search不可用，我将基于知识直接撰写综述。

- Write(attention_optimization_survey.tex)
 - └ Wrote 254 lines to attention_optimization_survey.tex
 - 1 \documentclass[UTF8]{ctexart}
 - 2 \usepackage[utf8]{inputenc}
 - 3 \usepackage{amsmath,amssymb}
 - 4 \usepackage{graphicx}
 - 5 \usepackage{hyperref}

将 Claude Code 变成专业的 Agent

让 Claude Code 写一篇综述

- 初次给定 prompt:
 - 围绕机器学习, 注意力机制, 神经网络优化等领域, 展开调研, 写成一篇综述, tex 大概不要写太短, 必须有 bibtex
- 右图问题:
 - 搜索工具失效
 - 在没有搜索结果的情况下依然继续写综述

解决方式:

- 说清楚需要在 arxiv 上搜索文献
- 不要使用 Web Search 工具

围绕机器学习, 注意力机制, 神经网络优化等领域, 展开调研, 写成一篇综述, tex 大概不要写太短, 必须有 bibtex

- 我来为你调研并撰写一篇关于机器学习、注意力机制和神经网络优化的综述。首先让我检查一下当前环境和 memory。

Read 1 file, listed 1 directory (ctrl+o to expand)

- Web Search("attention mechanism neural network deep learning survey 2024 2025")
 - └ Did 0 searches in 118ms

- Web Search("neural network optimization methods survey Adam optimizer 2024")
 - └ Did 0 searches in 105ms

- Web Search("transformer attention mechanism self-attention recent advances 2025")
 - └ Did 0 searches in 78ms

- Web search不可用, 我将基于知识直接撰写综述。

- Write(attention_optimization_survey.tex)
 - └ Wrote 254 lines to attention_optimization_survey.tex
 - 1 \documentclass[UTF8]{ctexart}
 - 2 \usepackage[utf8]{inputenc}
 - 3 \usepackage{amsmath,amssymb}
 - 4 \usepackage{graphicx}
 - 5 \usepackage{hyperref}

将 Claude Code 变成专业的 Agent

让 Claude Code 用自带的工具写一篇综述

- 优化提示词，明确指出需要 arxiv，以及不能使用 Web Search
- 右图 Agent 懂得通过网络搜索 arxiv 关键词并读取摘要
- 最后结果
 - 45 篇文献的 references.bib
 - 一个综述 tex 文档

Tips :

- Claude Code 知道如何在 arxiv 搜索，即便没有提供特制的工具

围绕机器学习，注意力机制，神经网络优化等领域，展开调研，写成一篇综述，tex 大概不要太短，必须有 bibtex。

你需要在 arxiv 上搜索相关的文章，最好能包含2026最新的文献，不能跳过检索直接写综述
内置的 Web Search 是无效的，请你想其他办法

● 我将使用 WebFetch 工具直接访问 arXiv 来搜索相关论文。

● Fetch(https://arxiv.org/search/?searchtype=all&query=attention+mechanism+neural+network+optimization&start=0)
└ Received 252.7KB (200 OK)

● Fetch(https://arxiv.org/search/?searchtype=all&query=attention+mechanism+neural+network+optimization&start=0)
└ Received 253.7KB (200 OK)

● Fetch(https://arxiv.org/search/?searchtype=all&query=attention+mechanism+neural+network+optimization&start=0)
└ Received 254.1KB (200 OK)

Fetch(https://arxiv.org/search/?searchtype=all&query=attention+mechanism+neural+network+optimization&start=0)
└ Fetching...

● Fetch(https://arxiv.org/abs/2604.01757)
└ Received 47.1KB (200 OK)

Muon²: Boosting Muon via Adaptive Second-Moment Preconditioning

Ziyue Liu, Ruijie Zhang, Zhengyang Wang, Yequan Zhao, Yupeng Su, Zi Yang, Zheng Zhang

Showing 1-50 of 70 results for all: attention mechanism survey transformer

1. arXiv:2604.01757 [pdf, ps, other] [math.NA](#)
Attention Mechanisms Through the Lens of Numerical Methods: Approximation Methods and Alternative Formulations
Authors: Michel Fabrice Serret, Alice Cortinovis, Yijun Dong, Diana Halikias, Anna Ma, Fabio Matti, Deanna Needell, Katherine J. Pearce, Elizaveta Rebrova, Disha Shur, Rudi Smith, Hai-Xiao Wang, Laura Grigori
Abstract: The attention...
Submitted 2 April, 2026; originally announced April 2026.
Comments: 41 pages, 14 figures

2. arXiv:2602.14082 [pdf, ps, other] [gr-qc](#)
The Theoretical Landscape of Mimetic Gravity: A Comprehensive Review
Authors: O. Malaeab
Abstract: ...and a scalar field, the theory naturally gives rise to a longitudinal degree of freedom that mimics the behavior of cold dark matter. This review reviews a comprehensive survey of the

结果展示

可以看出前面 tex 编译乱掉了

2026	
Abstract	
Transformer	SGD Adam Muo

1

2017 Vaswani et al. [2017] Transformer	
of-the-art	SGD Adam [Kingma and Ba, 2014]
2024] Sophia[Liu et al., 2023]	Adam

2 Transformer

2.1

Self-Attention	$X \in \mathbb{R}^{n \times d}$	$Q = XW_Q$ $K = XW_K$ $V = XW_V$	
Attention(Q, K, V) = softmax		$\left(\frac{QK^T}{\sqrt{d_k}}\right) V$	(1)
n d_k /	QK^T	$O(n^2d)$	Serret et al. [2025]
Fichtl et al. [2025]	Transformer		

参考文献是正常的，有 44 篇

References

Anonymous. Muon: Matrix updates for large-scale neural network training. *arXiv preprint*, 2024.

Minqi Chen, Zhongqi Yue, Shihao Zhang, Yun Xu, Peng Wu, Kaixiang Xu, Zeyi Huang, and Hanwang Zhang. Efficient matrix implementation for rotary position embedding. *arXiv preprint arXiv:2604.09742*, 2025.

Ranjith Chodavarapu and Lei Xu. The illusion of equivalence: Systematic fp16 divergence in kv-cached autoregressive inference. *arXiv preprint arXiv:2604.15409*, 2025.

Rewon Chordia. Efficient content-based sparse attention with transformers. *arXiv preprint arXiv:2103.05893*, 2021.

Dejan Cugalj and Aleksandar Jevremovic. Cawn: Continuous acoustic wave networks for autoregressive language modeling. *arXiv preprint arXiv:2604.04250*, 2025.

Steffen Derieich, Thang Do, and Arnulf Jentzen. Uniform a priori bounds and error analysis for the adam stochastic gradient descent optimization method. *arXiv preprint arXiv:2603.18899*, 2025a.

Steffen Derieich, Robin Graeber, Arnulf Jentzen, and Adrian Riekert. Asymptotic stability properties and a priori bounds for adam and other gradient descent optimization methods. *arXiv preprint arXiv:2509.10476*, 2025b.

Alexander M Fichtl, Jeremias Bohn, Josefin Kelber, Edoardo Mosca, and Georg Groh. The end of transformers? on challenging attention and the rise of sub-quadratic architectures. *arXiv preprint arXiv:2510.05364*, 2025.

将 Claude Code 变成专业的 Agent

让 Claude Code 写一篇可编译的综述

- 依旧直接给 Claude Code 提示词就好，不用关心具体的问题（前提是安装了 latex）
- 右图显示修复的很顺利
- 那么这次的经验如何保留到以后呢？

回顾我们刚才学过的：

- Claude Code 可以将这份经历提取成 skill

```
请你修正你的 tex，确保编译通过

Read 1 file (ctrl+o to expand)

● Bash(pdflatex --version 2>/dev/null || echo "pdflatex not found")
└─ pdfTeX 3.141592653-2.6-1.40.27 (TeX Live 2025)
   kpathsea version 6.4.1
   Copyright 2025 Han The Thanh (pdfTeX) et al.
   ... +9 lines (ctrl+o to expand)

● Bash(pdflatex -interaction=nonstopmode attention
head -100)
└─ This is pdfTeX, Version 3.141592653-2.6-1.40.
   d format=pdflatex)
   restricted \write18 enabled.
   ... +68 lines (ctrl+o to expand)

● 中文支持问题。需要添加ctex包来处理中文字符。

● Update(attention_optimization_survey.tex)
└─ Added 1 line
   1 \documentclass[12pt,a4paper]{article}
   2 \usepackage[utf8]{inputenc}
   3 +\usepackage{ctex}
   4 \usepackage{amsmath,amssymb}
   5 \usepackage{graphicx}
   6 \usepackage{margin=1in}{geometry}
```

注意力机制与神经网络优化研究综述

2026

Abstract

注意力机制和神经网络优化是深度学习领域的两个核心研究方向。近年来，随着 Transformer 架构的兴起，注意力机制成为序列建模的主导方法，但其二次复杂度问题催生了大量高效注意力变体的研究。与此同时，神经网络优化技术也在不断发展，从经典的 SGD、Adam 到最新的 Muon 等二阶方法，优化器的设计持续演进。本文系统综述了注意力机制与神经网络优化领域的研究进展。在注意力机制方面，我们详细介绍了标准自注意力的计算复杂度问题及其解决方案，涵盖线性注意力、稀疏注意力、状态空间模型等主要技术路线。在优化器方面，我们回顾了从一阶方法到自适应二阶方法的演进历程，分析了最新提出的 Muon、Adam 等优化器的理论基础与实践表现，并探讨了优化器的选择对神经网络收敛特性的影响。最后，我们讨论了这两个领域的交叉应用，包括注意力机制在优化器设计中的应用以及优化目标驱动的注意力架构创新。本综述旨在为研究者和工程师提供关于这两个领域的全面技术参考。

1 引言

深度学习在计算机视觉、自然语言处理、语音识别等领域取得了突破性进展，其核心驱动力在于神经网络架构的创新与训练技术的进步。注意力机制和优化器设计作为深度学习训练的两个关键环节，深刻影响着模型的表达能力、训练效率和最终性能。

注意力机制的概念最初在序列到序列模型中被提出，用于解决编码器-解码器之间的信息传递问题。2017 年，Vaswani et al. [2017] 提出了 Transformer 架构，完全基于注意力机制实现序列建模，摒弃了循环神经网络中的递归计算。这一创新使得模型能够并行处理序列中的所有位置，显著提升了训练效率，并在机器翻译、文本生成、图像分类等任务上取得了 state-of-the-art 结果。然而，标准自注意力的计算复杂度与序列长度的平方成正比，当处理长序列时，计算成本成为制约模型扩展的主要瓶颈。

神经网络优化器负责确定参数更新的方向和步长，直接影响模型的收敛速度和最终性能。自随机梯度下降 (SGD) 被提出以来，研究者不断探索更有效的优化算法。Adam 优化器 [Kingma and Ba, 2014] 凭借其自适应学习率特性成为深度学习训练的事实标准，然而近年来，一些新方法如 Muon [Anonymous, 2024]、Sophia [Liu et al., 2023] 等的提出对 Adam 的主导地位提出了挑战。

本文系统综述了注意力机制与神经网络优化领域的研究进展。第二节介绍注意力机制的演进，第三节讨论神经网络优化器的最新发展，第四节探讨两个领域的交叉应用，第五节总结全文并展望未来研究方向。

将 Claude Code 变成专业的 Agent

通过 skill-creator，claude code 将这次的方法论永久的保存

```
/skill-creator:skill-creator 请你将我们写 survey  
的方法与成 skill 保存下来，以后任何需要 arxiv 检索论文信息，生成 pdf survey  
的时候都可以复用
```

● 我来分析我们刚才做 survey 的完整工作流程，然后将其封装成一个可复用的 skill。

工作流程回顾：

1. 在 arXiv 上搜索相关主题论文
2. 用 WebFetch 获取每篇论文的标题、作者、摘要、arXiv ID
3. 整理文献引用信息为 BibTeX 格式
4. 撰写 LaTeX 综述论文（中文），包含引言、注意力机制、优化器等章节
5. 使用 xelatex + xeCJK + fontspec 编译支持中文的 PDF

将 Claude Code 变成专业的 Agent

当再次执行类似的任务，只需要简单的提示即可

- 运行稳定之后，可以
- `$ claude -p " 请你写一篇有关大模型混合精度训练最新进展的综述论文 "`
`--permission-mode bypassPermissions`

请写一篇有关大模型混合精度训练的最新进展的综述

- Skill(arxiv-survey)
 - └ Successfully loaded skill
- 我来为您生成关于大模型混合精度训练最新进展的综述论文。首先搜索相关论文。
- Fetch(<https://arxiv.org/search/?searchtype=all&query=mixed+precision+training+large+language+model&start=0>)
 - └ Fetching...
- Fetch(<https://arxiv.org/search/?searchtype=all&query=FP8+training+LLM+optimization&start=0>)
 - └ Fetching...
- Fetch(<https://arxiv.org/search/?searchtype=all&query=quantization+large+model+training+transformer&start=0>)
 - └ Fetching...

大语言模型混合精度训练研究综述

2026 年 4 月

Abstract

大语言模型 (Large Language Models, LLMs) 的参数规模快速增长, 对计算资源和内存开销提出了严峻挑战。混合精度训练通过动态选择不同数值精度, 显著降低训练成本同时保持模型性能。本文综述了 2024-2026 年大模型混合精度训练领域的最新研究进展, 涵盖 FP8 低精度训练、FP4 极致压缩、MoE 专家级混合精度、动态精度切换、内存高效优化器等多个方向。分析了各方法的技术特点、性能收益与适用场景, 并探讨了未来发展趋势与潜在研究方向。

1 引言

大语言模型的训练需要处理海量参数和梯度数据, 传统 FP32 全精度训练面临着内存占用大、计算成本高的困境。混合精度训练 (Mixed Precision Training) 应运而生, 其核心思想是根据不同变量 (如权重、梯度、优化器状态) 的数值特性, 动态选择最优精度格式, 在保证模型精度的同时最大化计算效率和内存利用率。

NVIDIA GPU 对 FP8 格式的原生支持使得低精度训练成为主流方向。Microsoft 提出的 FP8-LM 框架证明 FP8 训练可将 GPT-175B 模型内存减少 39%, 训练速度提升 75%。2025 年, Quartet 工作进一步将精度推向 FP4, 证明在 Blackwell 架构上全 FP4 训练可与 FP16 竞争。与此同时, 1-bit 量化 (BitNet b1.58) 的发展则探索了极致压缩的极限。

本文系统综述了混合精度训练的最新进展, 第 2 节介绍技术背景, 第 3 至 7 节分别涵盖 FP8 训练、FP4 与极致压缩、MoE 混合精度、动态精度切换、内存高效优化器方向, 第 8 节进行总结与展望。

对比传统搭建 Agent 和 Claude Code

传统 LangGraph 等

- I. 需要提示词精确规定 llm 的输出格式，从中提取参数
- I. 形态是一个有向图，用 python 代码控制，设计成本高
- I. 需要针对错误设计额外的纠错流程
- I. 效果非常稳定，但是不够灵活

Claude Code 等

- I. Claude Code 自己掌握流程，有稳定调用工具的能力
- I. 形态是一个不断调用工具的 loop，只需要提示词就能控制
- I. 具有自我纠错能力
- I. 随着模型能力加强，运行越来越稳定

```
agents
├── base.py
├── executor.py
├── __init__.py
├── planner.py
├── prompts.py
├── reflector.py
├── validator.py
└── config.py
  ├── __init__.py
  └── loop
    ├── controller.py
    ├── __init__.py
    ├── state_machine.py
    └── termination.py
main.py
memory
├── __init__.py
├── knowledge_graph.py
├── manager.py
├── session.py
└── vector_store.py
models
├── finding.py
├── __init__.py
├── paper.py
└── research_state.py
services
├── __init__.py
└── llm_client.py
survey
├── bibtex_formatter.py
├── generator.py
├── __init__.py
└── templates
   └── base.tex
tools
├── base.py
├── __init__.py
├── pdf
│   └── analyzer.py
```

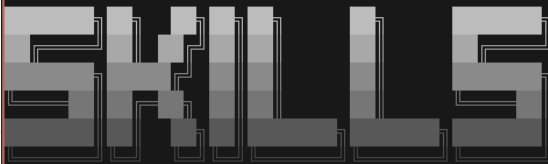
补充：如何 Agent 自主查找现有的成熟 skill

Install a Skill

```
npx skills add vercel-labs/agent-skills
```

或者直接跟 Claude Code 描述：
请用 npx skills 找 xxx

```
[daibo@Arch ~]$ npx skills find arxiv
```



```
Install with npx skills add <owner/repo@skill>
```

```
karpthy/nanochat@read-arxiv-paper 1.3K installs
```

```
└─ https://skills.sh/karpthy/nanochat/read-arxiv-paper
```

```
yorkeccak/scientific-skills@arxiv-search 800 installs
```

```
└─ https://skills.sh/yorkeccak/scientific-skills/arxiv-search
```

```
langchain-ai/deepagents@arxiv-search 746 installs
```

```
└─ https://skills.sh/langchain-ai/deepagents/arxiv-search
```

```
neolabhq/context-engineering-kit@mcp:setup-arxiv-mcp 346 installs
```

```
└─ https://skills.sh/neolabhq/context-engineering-kit/mcp:setup-arxiv-mcp
```

```
varde80/arxiv-zotero-obsidian@arxiv-search 167 installs
```

```
└─ https://skills.sh/varde80/arxiv-zotero-obsidian/arxiv-search
```

```
yrom/arxiv-paper-translator@arxiv-paper-translator 160 installs
```

```
└─ https://skills.sh/yrom/arxiv-paper-translator/arxiv-paper-translator
```

实验二

Claude Code SubAgent

——主 Agent 处理不过来时，可以分配给子智能体

Part
II

更稳定的 Survey Agent

- 运行之前的 skill 有时会发现生成的 tex 文件有引用的问题

2.2 大规模 FP8 训练的稳定性问题

Fishman 等人 [?] 首次将 FP8 训练扩展至 2 万亿 token 规模（此前方法仅能处理约 100B token），并识别出一个关键的不稳定性来源：SwiGLU 激活函数引起的异常值（outlier）放大效应。

研究发现，在扩展训练过程中，SwiGLU 激活函数会导致某些维度产生极端的异常值，这些异常值在 FP8 量化时造成严重的舍入误差，进而破坏训练稳定性。为此，作者提出 Smooth-SwiGLU 修改方案，通过对 SwiGLU 的门控线性变换进行平滑处理，有效抑制了异常值的生成。在 256 片 Intel Gaudi2 加速器上训练 7B 参数模型，使用 Smooth-SwiGLU 后实现了 34% 的吞吐量提升，同时与 BF16 基准的性能相匹配。

- 精读摘要的数量偏少，内容不够充实
- 写作质量不够高，缺少模板
- 单个 Agent 如果处理不了，可能需要做一些任务拆分

更稳定的 Survey Agent

arxiv-survey

skill有问题是工作量太大，读文章不够细节，写作效果也不好，请你充分利用子智能体，分配2-3个支持 webfetch

的子智能体从不同角度并行调研大模型混合精度训练的最新进展，完成各自段落的 tex 和引用文件，再由你整合成完整的文章，并派一个子智能体检查编译错误和文章质量。

●我来并行分配多个子智能体从不同角度调研大模型混合精度训练的最新进展。

Running 3 agents... (ctrl+o to expand)

└ 调研混合精度训练理论与算法进展 · 6 tool uses · 19.9k tokens

└ Fetch: <https://arxiv.org/search/?searchtype=all&query=FP...>

└ 调研混合精度训练系统工程进展 · 4 tool uses · 3.8k tokens

└ Fetch: <https://arxiv.org/search/?searchtype=all&query=Tr...>

└ 调研混合精度训练应用与评估进展 · 7 tool uses · 16.3k tokens

└ Fetch: <https://arxiv.org/abs/2410.19313>

(ctrl+b to run in background)

更稳定的 Survey Agent

2026 年 4 月 23 日

参考文献

- Various Authors. Convergence analysis of mixed precision training. *NeurIPS 2023*, 2023a.
- Various Authors. Gradient distribution and loss dynamics in low-precision training. *ICML 2023*, 2023b.
- Various Authors. Mixtral of experts. *Mistral AI*, 2023c.
- Various Authors. Stability analysis of low-precision training for large language models. *ICLR 2024*, 2024.
- Tianqi Chen, Bing Xu, and Jon Goodnough. Training deep nets with sublinear memory cost. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016.
- Tim Dettmers, Artid Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

1 引言

随着大语言模型 (LLM) 参数规模从十亿级增长至万亿级, 训练过程的计算成本与内存需求呈指数级上升。混合精度训练通过协调不同数值精度以兼顾训练效率与模型精度, 已成为大模型训练的核心技术范式。自 2017 年 NVIDIA 提出 FP16 与 FP32 混合训练方案以来, 该领域已从单一精度组合演进为涵盖 FP8、BF16、INT8、INT4 等多精度协同的复杂系统。

本文系统梳理混合精度训练在算法理论、系统工程与应用评估三个维度的最新进展。在算法理论层面, 重点涵盖 FP8 训练框架 Peng et al. [2023]、量化误差补偿机制 Frantar et al. [2023], Wang et al. [2023]、极低位宽训练 Lin et al. [2024] 以及缩放因子设计 Authors [2024]。在系统工程层面, 探讨分布式训练中的精度策略 Rajbhandari et al. [2020], Shoybi et al. [2019], Zhao et al. [2023a]、MoE 架构下的精度设计 Lepikhin et al. [2020], Jiang et al. [2024]、内存优化技术 Chen et al. [2016], Rajbhandari et al. [2022], Dettmers et al. [2023] 以及异构 GPU 集群支持 Harlap et al. [2018], Zhang et al. [2023]。在应用评估层面, 分析不同精度格式的性能对比 Touvron et al. [2023], Authors [2023a,c]、收敛性分析与稳定性保障机制。

2 算法理论进展

混合精度训练的理论基础建立在数值精度与计算效率的权衡之上。NVIDIA 提出的原始混合精度方案 Micchelli et al. [2017] 将 FP16 用于前向与反向传播以利用 Tensor Core 加速, 而权重更新保持 FP32 以维持数值稳定性, 在 V100 GPU 上实现约 3 倍吞吐量提升。然而, 传统 FP16 在大规模模型训练中存在着显著稳定性问题, 研究表明 FP16 训练在 7B 参数以上模型中出現约 42% 的概率崩溃 Authors [2024]。

2.1 FP8 混合精度训练框架

FP8 格式凭借其在 Hopper 架构上的原生支持, 成为当前混合精度训练的研究焦点。NVIDIA 提出的 FP8-LM 框架 Peng et al. [2023] 首次系统性地构建了面向 LLM 的 FP8 混合精度训练架构。该框架设计了延迟梯度缩放与动态精度路由机制, 针对激活分布动态选择 E4M3 或 E5M2 格式。实验数据显示, 在 175B 参数 GPT 模型上, FP8 训练相比 BF16 实现 39% 内存降低, 训练吞吐量提升 75%, 而 MMLU、HellaSwag 等基准精度差异小于 0.5%, 验证了千亿参数规模下的有效性。

实验三

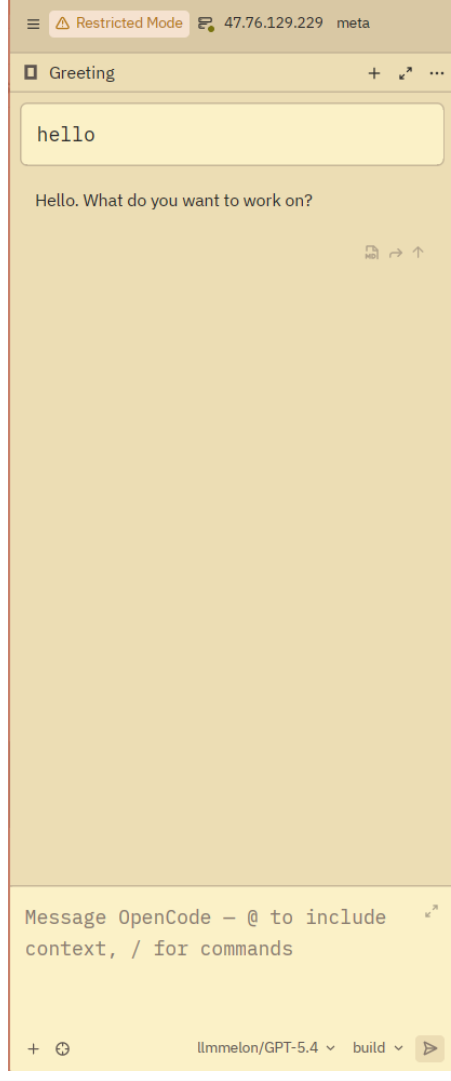
基于 ACP 的多智能体

使用 ACP 协议接入不同架构的 Agent

Part
III

ACP 介绍

- ACP（全名 Agent Client Protocol）是一款 zed 编辑器发起的用于 Agent 和编辑器沟通的统一协议
- 常见操作：
 - 会话管理（新建、列表、加载、取消）
 - 发送提示词、接收智能体相应、处理工具许可
 - 切换模型、参数等
- 对 Agent 而言：ACP 使得智能体有机会以“使用者”的身份使用另一个智能体（比如让 Claude Code 有机会操作 Codex）
- 核心是 acpx 工具



ACPX 介绍

- ACPX 是一款终端工具，通过 acp 协议调用常见的智能体
- 通过 `npx acpx@latest --skill install acpx` 可以让任何支持 skill 和 bash 的智能体拥有调用 cc, codex 等顶尖智能体的能力
- 相比于 `claude -p xxx`，acpx 有以下优点
 - 流式输出中间状态，对长时间运行的 Agent 比较友好
 - 可读性高，原始输出就能被 agent 理解和监控
 - 支持会话持久化保存和恢复
 - 一套方法无缝衔接各种支持 acp 的智能体
- 相比 Subagent: 不用额外的配置

运行截图

```
[daibo@Arch claude-acpx]$ acpx claude exec "hello"
```

```
[client] initialize (running)
```

```
[client] session/new (running)
```

```
[thinking] The user just said "hello". I should respond with a friendly  
Hello! How can I help you today?
```

```
[done] end_turn
```

```
[daibo@Arch ~]$ acpx codex exec "hello"
```

```
[client] initialize (running)
```

```
[client] authenticate (running)
```

```
[client] session/new (running)
```

```
Hello! What can I help you with?
```

```
[done] end_turn
```

```
[daibo@Arch claude-acpx]$ acpx claude sessions new
```

```
[acpx] created session cwd (6b2ee1b8-25df-4487-ad06-8c59dacdff0b)
```

```
[acpx] agent: claude
```

```
[acpx] cwd: /home/daibo/claude-acpx
```

```
[daibo@Arch ~]$ acpx claude sessions list
```

```
6b2ee1b8-25df-4487-ad06-8c59dacdff0b - /home/daibo/claude-acpx 2026-04-23T0  
1:54:11.493Z
```

```
4d5ab800-a157-4522-9f90-1c6db6b2bf35 [closed] acp:claude:6c5f7b07-c5ab-4084-afa6-7  
e45c7b7ae71 /mnt/data/fat/reaslab-agent 2026-04-02T05:49:59.241Z
```

三种多智能体方案对比

方案	集成成本	控制粒度	适用任务	与前两部分衔接
Subagent	低	中	同构任务拆分	直接复用 Part I 的角色
Codex MCP	中	细	外部 specialist 协作	让 Part II 的编排接入异构 agent
ACPX	高	最细	长流程科研系统	把 Part I + II 提升为协议级系统

选择建议

能用 Python 编排先别急着上多智能体；只有当分工、并行或异构模型带来真实收益时，再升级协作机制。

实验四

Python 控制

利用 Claude Agents SDK 深度定制 Agent

Part
III

Claude Agent SDK 介绍

- Claude Agents SDK 本质是 Claude Code 在 Python 里的简单封装，得益于 Claude Code 良好的基础设施，简单的代码就能写出良好的 Agent
- 安装方式：`pip install claude-agent-sdk`
- 小提示：claude code plugin 有 agent-sdk-dev 用于开发 Agent
- 以下是一个简单的例子

```
from claude_agent_sdk import query, ClaudeAgentOptions

options = ClaudeAgentOptions(
    allowed_tools=["Read", "Edit", "Bash"],
    system_prompt="你是一个代码审查专家",
    max_turns=10
)

async for message in query(prompt="找出所有潜在的安全漏洞", options=options):
    print(message)
```

Claude Agent SDK 介绍：自定义 MCP 工具

```
@tool("add_todo", "添加待办事项", {"title": str, "priority": str})
async def add_todo(args):
    todos.append({"title": args["title"], "priority": args.get("priority", "medium")})
    return {
        "content": [
            {
                "type": "text",
                "text": f"已添加: [{args.get('priority', 'medium')}] {args['title']}",
            }
        ]
    }
```


Claude Agent SDK 介绍：将工具注册到 Agent

```
todo_server = create_sdk_mcp_server(  
    name="todo-tools",  
    version="1.0.0",  
    tools=[add_todo, list_todos, search_docs],  
)
```

```
options = ClaudeAgentOptions(  
    system_prompt="你是一个中文助手。使用提供的工具来管理待办事项和搜索文档。  
。",  
    mcp_servers={"todo": todo_server},  
    allowed_tools=["mcp__todo__add_todo", "mcp__todo__list_todos", "mcp__to  
o__search_docs"],  
    max_turns=10,  
)
```

Claude Agent SDK 介绍：添加 Hooks

```
hooks_config = {
    "PreToolUse": [
        HookMatcher(
            matcher=None,
            hooks=[pre_tool_use_audit],
        ),
        HookMatcher(
            matcher="Bash",
            hooks=[pre_tool_use_block_bash],
        ),
    ],
}
```

```
async def pre_tool_use_block_bash(
    input: HookInput,
    tool_use_id: str | None,
    context: HookContext
) -> HookJSONOutput:
```

Hook 返回值

- └─ additionalContext →  Claude 能读到 (上下文注入)
- └─ systemMessage →  Claude 能读到 (system 级别)
- └─ permissionDecisionReason →  Claude 能读到 (deny 的理由)
- └─ updatedInput →  Claude 看到改后的参数 (不知道被改过)
- └─ decision: "block" →  Claude 只知道"工具调用失败", 不知道是被 Hook 拦的
- └─ 返回 {} →  Claude 完全感知不到这个 Hook 存在

Claude Agent SDK 介绍：添加 Skill

```
options = ClaudeAgentOptions(  
    system_prompt="你是一个开发助手，使用中文回答。  
    skills=["brainstorming"],  
    permission_mode="bypassPermissions",  
    max_turns=5,  
)
```

Claude Agent SDK 介绍：添加 SubAgent

```
options = ClaudeAgentOptions(  
    system_prompt="你是调度助手，委派任务给子智能体。",  
    agents={  
        "worker": AgentDefinition(  
            description="通用工作智能体",  
            prompt="你是工作助手，按要求完成任务。用中文回答。",  
            maxTurns=3,  
        ),  
    },  
    permission_mode="bypassPermissions",  
    max_turns=10,  
)
```

多智能体总结

Part V

方案一： Claude Code 自带 Subagent

机制

让 Claude Code 在同一体系内拆分任务，交给 subagent 执行。

优点

内建、轻量、心智负担低
适合同构任务拆分

局限

角色能力还是主要受 Claude 体系限制
跨系统协作能力有限

适合的协作方式

Coordinator

- > survey subagent
- > experiment subagent
- > writing subagent

场景

- 统一工具栈
- 统一权限模型
- 轻量任务并行或拆分

方案二： ACPX

机制

显式 agent-to-agent 通信，
强调会话、队列、权限和协议
层编排。

优点

可做更复杂的多轮、多角色、
多会话协作
适合更像“系统”的场景

局限

系统设计成本更高
更适合作为高阶内容而非入门
默认选项

适合理解为

不是“再多几个 prompt”
而是显式的 agent runtime。

适合

- 长流程科研任务
- 明确的消息边界
- 队列、权限、状态管理
- 更复杂的 coordinator 逻辑

方案三： Claude Code Agents SDK

机制

通过 Python 自由配置
Claude Code

优点

Python 生态、自定义能力强

局限

需要一定的代码能力，不那么
开箱即用

适理解解为

Claude Code 的 Python 适配版

适合

- 嵌入已有的工作流
- 更灵活的自定义空间
- 批量任务

Q & A

感谢聆听 · 欢迎提问

